

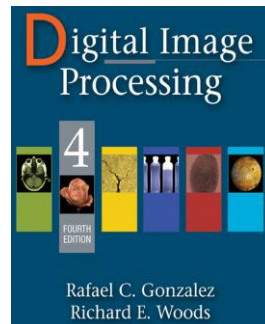


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گزارش تمرین اول - تصویرپردازی رقمی

بررسی نمونه‌هایی از سیگنال‌های دوبعدی و تبدیل فوریه آن‌ها



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،

Aminkianiworkeng@gmail.com

استاد درس: دکتر نقش نیلچی

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

۳	مستندات.....
۳	۱- مسئله و تحلیل کلی آن:.....
۴	۲- توابع مورد بررسی:.....
۸	۳- روش پیاده‌سازی:.....
۹	۴- کد اجرا به زبان پایتون:.....
۱۳	۵- نتایج:.....
۱۷	۶- مراجع.....

مستندات

۱- مسئله و تحلیل کلی آن:

در پردازش تصویر دیجیتال (بر اساس کتاب Digital Image Processing)، تحلیل **سیگنال‌های دوبعدی و تبدیل فوریه آن‌ها** نقش اساسی در درک رفتار سیستم‌های تصویربرداری دارد. پس بسیاری از سیستم‌های تصویربرداری را می‌توان با یک تابع پاسخ ضربه دوبعدی یا همان **PSF (Point Spread Function)** مدل کرد که نشان می‌دهد اگر یک نقطه‌ی ایده‌آل وارد سیستم شود، خروجی سیستم چگونه در فضا پخش می‌شود. تبدیل فوریه‌ی دوبعدی PSF را **OTF (Optical Transfer Function)** می‌نامند که نشان می‌دهد سیستم در حوزه‌ی فرکانس چگونه رفتار می‌کند. یعنی:

- **PSF (Point Spread Function)**: پاسخ سیستم در حوزه مکان (Spatial Domain)

- **OTF (Optical Transfer Function)**: تبدیل فوریه PSF در حوزه فرکانس

$$OTF(u, v) = \mathcal{F}\{PSF(x, y)\}$$

قرارداد تبدیل فوریه‌ی پیوسته:

$$F(u, v) = \iint f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

اگر تابع در حوزه‌ی مکان تیز و دارای لبه‌های ناگهانی باشد، انرژی بیشتری در فرکانس‌های بالا خواهد داشت. در مقابل، توابع نرم و هموار مانند Gaussian عمدتاً در فرکانس‌های پایین متمرکز هستند.

در بخش‌های بعدی، پنج تابع دوبعدی بررسی می‌شوند:

۲- توابع مورد بررسی:

۲-۱ Rectangle (تابع مستطیلی)

یک تابع مستطیلی دوبعدی که فقط در یک ناحیه‌ی محدود غیرصفر است.

$$f(x, y) = \begin{cases} 1 & |x| \leq a, |y| \leq b \\ 0 & otherwise \end{cases}$$

- تبدیل فوری:

$$F(u, v) = (2a \cdot \text{sinc}(2au))(2b \cdot \text{sinc}(2bv))$$

که در آن:

$$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

- نتیجه:

لبه‌ی تیز در حوزه‌ی مکان، به پخش شدن انرژی در فرکانس‌های بالا منجر می‌شود.

(لبه تیز → فرکانس بالا → ایجاد sinc)

۲-۲ Pyramid (هرم)

می‌توان به صورت حاصل ضرب دو تابع مثلثی یک‌بعدی نوشت:

$$f(x, y) = \max\left(1 - \frac{|x|}{a}, 0\right) \cdot \max\left(1 - \frac{|y|}{b}, 0\right)$$

$$f(x, y) = (1 - |x|/a)(1 - |y|/b)$$

این تابع در واقع از کانولوشن یک Rectangle با خودش به دست می‌آید. بنابراین در حوزه‌ی فرکانس، تبدیل آن برابر مربع پاسخ مستطیلی است:

- تبدیل فوریه:

$$F(u, v) = \text{sinc}^2$$

$$F(u, v) \propto \text{sinc}^2(2au) \text{sinc}^2(2bv)$$

- نتیجه:

نرم‌تر از rectangle → فرکانس کمتر

۲-۳ Gaussian (گوسی)

$$f(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- تبدیل فوریه:

$$F(u, v) = e^{-2\pi^2\sigma^2(u^2+v^2)}$$

- نتیجه:

تنها تابعی که در هر دو حوزه شکلش حفظ می‌شود و این خاصیت در پردازش تصویر و فیلترهای پایین‌گذر بسیار مهم است.

۲-۴-Peak (پیک)

به صورت شعاعی و با رفتار معکوس فاصله تعریف می‌شود:

$$f(x, y) = \frac{1}{r}$$

$$r = \sqrt{x^2 + y^2}$$

این تابع در مبدأ تکین است، بنابراین در پیاده‌سازی عددی باید با یک مقدار کوچک ϵ آن را منظم‌سازی کرد:

$$f(x, y) = \frac{1}{\max(r, \epsilon)}$$

- تبدیل فوریه:

این نوع تابع رفتار معکوس با فرکانس شعاعی دارد. در اینجا شکل کلی مهم‌تر از ثابت دقیق است، زیرا ثابت به قرارداد تبدیل فوریه و نرمال‌سازی بستگی دارد.

$$F(u, v) \propto \frac{1}{f}$$

- نتیجه:

- واگرا در مرکز
- انرژی زیاد در فرکانس پایین

۲-۵- Exponential Decay (کاهش نمایی)

این تابع یک پاسخ نرم و هموار دارد.

$$f(x, y) = e^{-ar}$$

- تبدیل فوریه:

$$F(u, v) = \frac{2\pi a}{(u^2 + v^2 + a^2)^{3/2}}$$

این تابع در حوزه‌ی فرکانس نیز شعاعی است و به صورت یک فیلتر پایین‌گذر رفتار می‌کند.

- نتیجه:

- ایزوتروپیک (دایره‌ای)
- مناسب مدل‌سازی blur

* **Rectangle** دارای مرزهای تیز است، پس فرکانس‌های بالاتر را بیشتر تحریک می‌کند .

* **Pyramid** نسبت به Rectangle نرم‌تر است، بنابراین طیف فرکانسی آن متمرکزتر و فشرده‌تر است .

* **Gaussian** هموارترین تابع این مجموعه است و در فرکانس نیز به صورت Gaussian باقی می‌ماند .

* **Peak** به دلیل تکینگی در مبدأ نیازمند منظم‌سازی عددی است .

* **Exponential Decay** تابعی شعاعی و نرم است که طیف آن نیز شکل فرکانسی ملایمی دارد.

۳- روش پیاده‌سازی:

نقشه‌ی راه به صورت زیر است:

۱. تولید grid دوبعدی

۲. تعریف هر تابع در حوزه مکان

۳. محاسبه FFT دوبعدی

۴. انتقال مرکز طیف به وسط تصویر با fftshift

۵. قدرمطلق طیف را رسم می‌کنیم.

۶. برای نمایش بهتر، از نمایش لگاریتمی استفاده می‌کنیم.

۷. نمایش :

○ Spatial Domain (شکل تابع در حوزه‌ی مکان)

○ Frequency Domain (قدرمطلق تبدیل فوریه در حوزه‌ی فرکانس، به صورت

لگاریتمی برای نمایش بهتر)

رفتار در فرکانس	رفتار در مکان	تابع
sinc	لبه تیز	Rectangle
sinc^2	نرم‌تر	Pyramid
Gaussian	نرم	Gaussian
$1/f$	شدید در مرکز	Peak
smooth low-pass	شعاعی	Exponential

۴- کد اجرا به زبان پایتون:

مرحله ۱: نصب پکیج‌ها

```
!pip -q install numpy scipy matplotlib
```

مرحله ۲: کد اصلی (اجرا در گوگل کولب)

- استفاده از `fftshift` برای انتقال فرکانس صفر به مرکز تصویر ضروری است .

- استفاده از `log10p` برای نمایش بهتر طیف فرکانسی بسیار مفید است.

```
import numpy as np
import matplotlib.pyplot as plt

try:
    from scipy.fft import fft2, fftshift
except Exception:
    from numpy.fft import fft2, fftshift

def make_grid(n=512, extent=8.0):
    x = np.linspace(-extent, extent, n, endpoint=False)
    y = np.linspace(-extent, extent, n, endpoint=False)
    X, Y = np.meshgrid(x, y)
    dx = x[1] - x[0]
    dy = y[1] - y[0]
    return X, Y, dx, dy, x, y

def rect2d(X, Y, a=1.5, b=1.0):
    return ((np.abs(X) <= a) & (np.abs(Y) <= b)).astype(float)

def pyramid2d(X, Y, a=2.5, b=2.5):
    Tx = np.maximum(1.0 - np.abs(X) / a, 0.0)
    Ty = np.maximum(1.0 - np.abs(Y) / b, 0.0)
    return Tx * Ty

def gaussian2d(X, Y, sigma=1.0):
    return (1.0 / (2.0 * np.pi * sigma**2)) * np.exp(-(X**2 + Y**2) / (2.0 * sigma**2))
```

```

def peak2d(X, Y, eps=1e-3):
    r = np.sqrt(X**2 + Y**2)
    return 1.0 / np.maximum(r, eps)

def exp_decay2d(X, Y, alpha=1.2):
    r = np.sqrt(X**2 + Y**2)
    return np.exp(-alpha * r)

def fft2_continuous_approx(f, dx, dy):
    return fftshift(fft2(np.asarray(f))) * dx * dy

def freq_axes(n, dx, dy):
    fx = np.fft.fftshift(np.fft.fftfreq(n, d=dx))
    fy = np.fft.fftshift(np.fft.fftfreq(n, d=dy))
    return fx, fy

def analytic_rectangle_ft(U, V, a=1.5, b=1.0):
    return (2 * a) * np.sinc(2 * a * U) * (2 * b) * np.sinc(2 * b * V)

def analytic_gaussian_ft(U, V, sigma=1.0):
    return np.exp(-2.0 * (np.pi**2) * (sigma**2) * (U**2 + V**2))

def analytic_peak_ft(U, V, eps=1e-12):
    Rf = np.sqrt(U**2 + V**2)
    return 1.0 / np.maximum(Rf, eps)

def analytic_exp_decay_ft(U, V, alpha=1.2):
    return (2.0 * np.pi * alpha) / np.power(U**2 + V**2 + alpha**2, 1.5)

def plot_pair(ax1, ax2, spatial, spectrum, x, y, fx, fy, title1, title2,
cmap="viridis"):
    extent_xy = [x.min(), x.max(), y.min(), y.max()]
    extent_f = [fx.min(), fx.max(), fy.min(), fy.max()]

    im1 = ax1.imshow(spatial, extent=extent_xy, origin="lower", cmap=cmap,
aspect="auto")
    ax1.set_title(title1)
    ax1.set_xlabel("x")

```

```

ax1.set_ylabel("y")
plt.colorbar(im1, ax=ax1, fraction=0.046, pad=0.04)

im2 = ax2.imshow(np.abs(spectrum), extent=extent_f, origin="lower",
cmap=cmap, aspect="auto")
ax2.set_title(title2)
ax2.set_xlabel("u")
ax2.set_ylabel("v")
plt.colorbar(im2, ax=ax2, fraction=0.046, pad=0.04)

def main():
    n = 512
    extent = 8.0
    X, Y, dx, dy, x, y = make_grid(n=n, extent=extent)
    U, V = np.meshgrid(*freq_axes(n, dx, dy))

    a, b = 1.5, 1.0
    sigma = 1.0
    alpha = 1.2
    eps = 1e-3

    rect = rect2d(X, Y, a=a, b=b)
    pyr = pyramid2d(X, Y, a=2.5, b=2.5)
    gauss = gaussian2d(X, Y, sigma=sigma)
    peak = peak2d(X, Y, eps=eps)
    decay = exp_decay2d(X, Y, alpha=alpha)

    F_rect = fft2_continuous_approx(rect, dx, dy)
    F_pyr = fft2_continuous_approx(pyr, dx, dy)
    F_gauss = fft2_continuous_approx(gauss, dx, dy)
    F_peak = fft2_continuous_approx(peak, dx, dy)
    F_decay = fft2_continuous_approx(decay, dx, dy)

    A_rect = analytic_rectangle_ft(U, V, a=a, b=b)
    A_gauss = analytic_gaussian_ft(U, V, sigma=sigma)
    A_peak = analytic_peak_ft(U, V, eps=1e-12)
    A_decay = analytic_exp_decay_ft(U, V, alpha=alpha)

    spectra = [
        (F_rect, "Rectangle"),
        (F_pyr, "Pyramid"),
        (F_gauss, "Gaussian"),
        (F_peak, "Peak"),
        (F_decay, "Exponential Decay"),

```

```

]

spatial_list = [rect, pyr, gauss, peak, decay]
titles_spatial = [
    f"Spatial domain: Rectangle (a={a}, b={b})",
    "Spatial domain: Pyramid",
    f"Spatial domain: Gaussian (sigma={sigma})",
    f"Spatial domain: Peak (1/r, regularized)",
    f"Spatial domain: Exponential Decay (alpha={alpha})",
]

titles_freq = [
    "Fourier magnitude: Rectangle",
    "Fourier magnitude: Pyramid",
    "Fourier magnitude: Gaussian",
    "Fourier magnitude: Peak",
    "Fourier magnitude: Exponential Decay",
]

fig, axes = plt.subplots(5, 2, figsize=(14, 28))
fig.suptitle("2D Signals and Their Fourier Transforms", fontsize=18, y=0.995)

for i, (sig, (Fnum, _name)) in enumerate(zip(spatial_list, spectra)):
    plot_pair(
        axes[i, 0],
        axes[i, 1],
        sig,
        Fnum,
        x,
        y,
        U[0, :],
        V[:, 0],
        titles_spatial[i],
        titles_freq[i],
    )

plt.tight_layout()
plt.show()

fig2, axes2 = plt.subplots(2, 2, figsize=(12, 10))
fig2.suptitle("Selected Analytic Frequency Responses", fontsize=16)

comps = [
    ("Rectangle: analytic FT", A_rect, axes2[0, 0]),
    ("Gaussian: analytic FT", A_gauss, axes2[0, 1]),
    ("Peak: reference 1/f", A_peak, axes2[1, 0]),

```

```

("Exponential Decay: analytic FT", A_decay, axes2[1, 1]),
]

for title, arr, ax in comps:
    im = ax.imshow(np.abs(arr), extent=[U.min(), U.max(), V.min(), V.max()],
origin="lower", aspect="auto")
    ax.set_title(title)
    ax.set_xlabel("u")
    ax.set_ylabel("v")
    plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()

main()

```

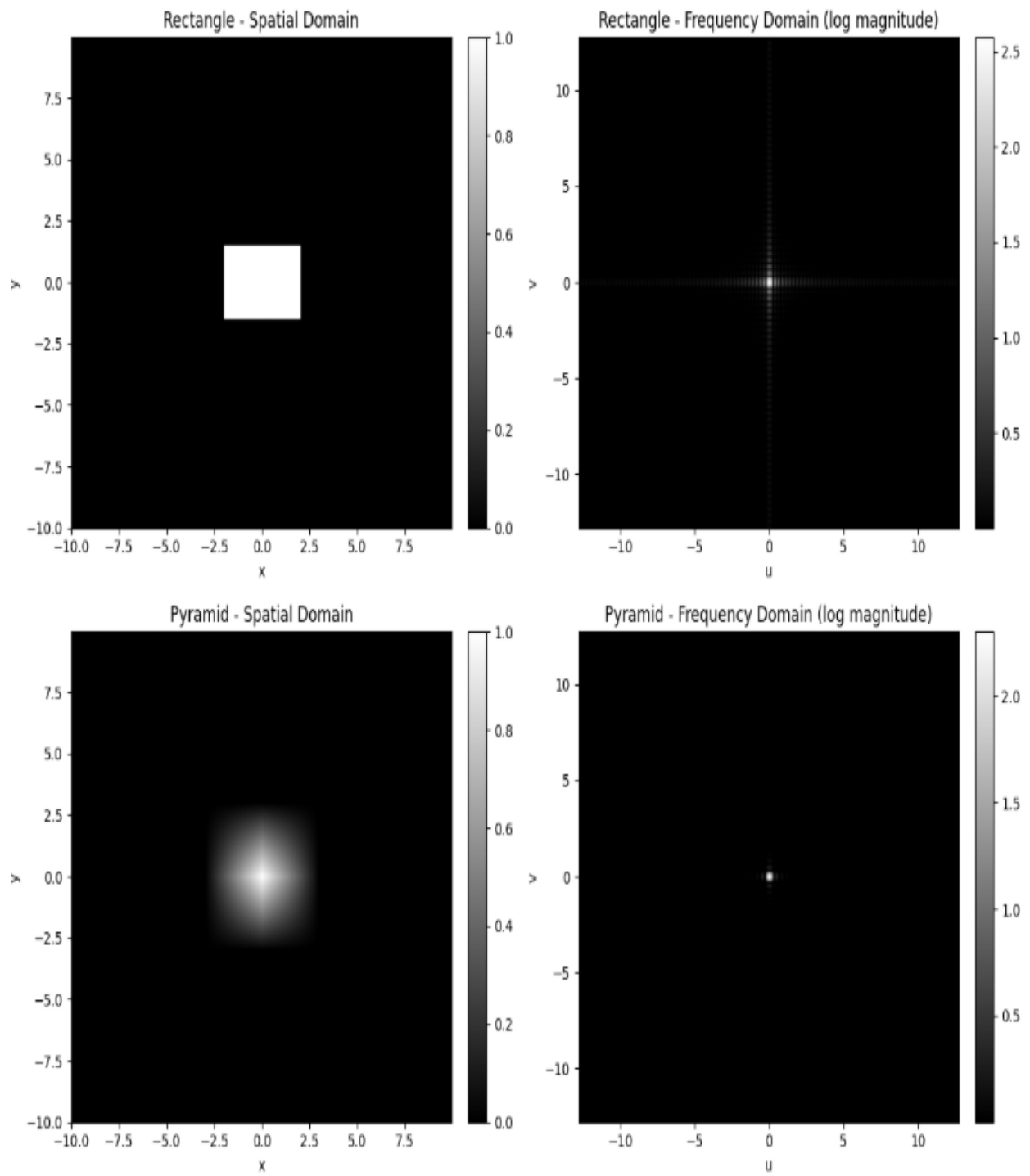
۵- نتایج:

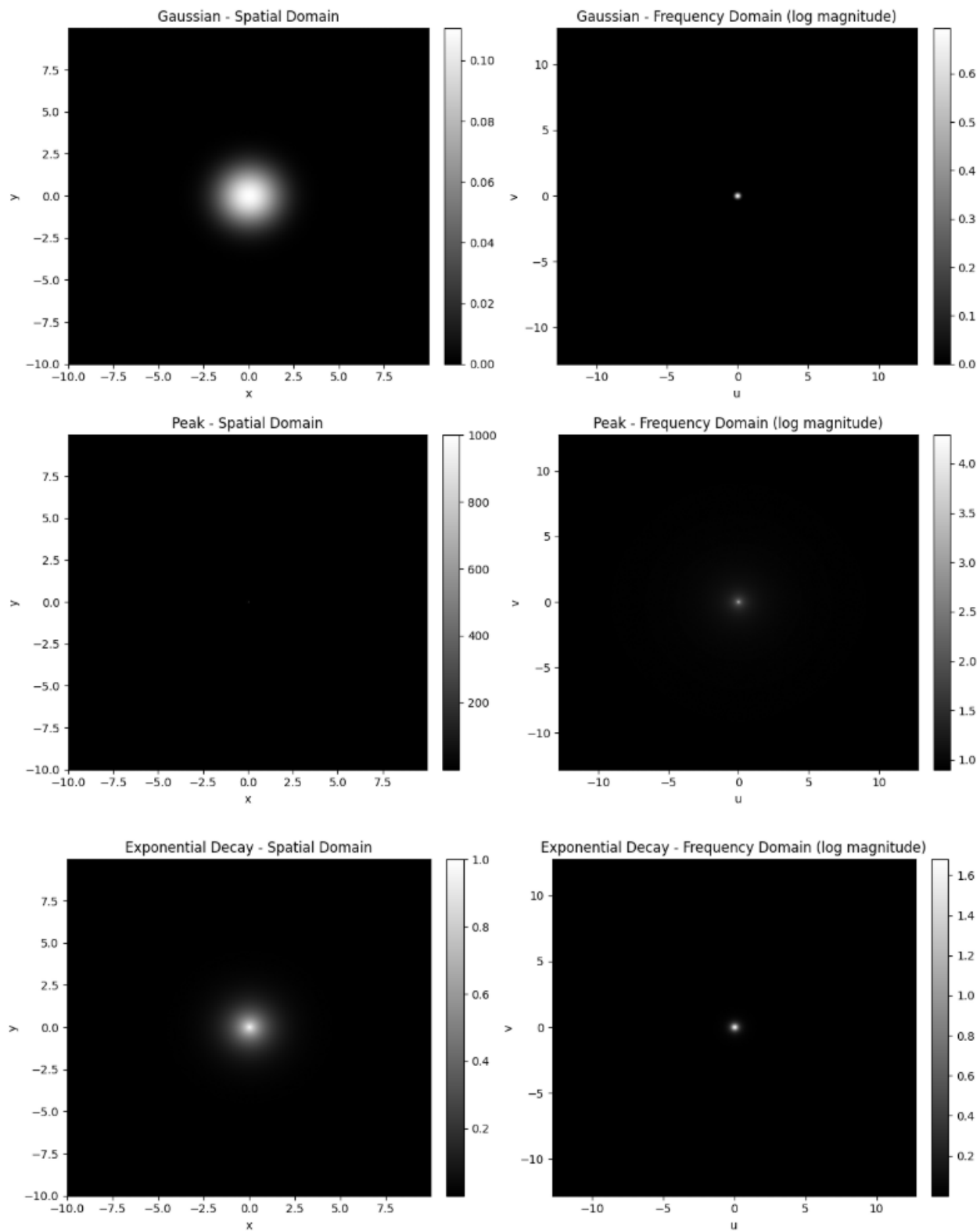
- لبه‌های تیز → فرکانس بالا
- توابع نرم → فرکانس پایین
- Gaussian بهترین رفتار فیلتر پایین گذر دارد.
- Fourier Transform ابزار اصلی تحلیل سیستم‌های تصویری است.

The screenshot shows a Google Colab notebook interface. On the left, there's a sidebar with a file explorer and a 'Release notes' panel. The main area displays a terminal window with the command `!pip install numpy matplotlib scipy` and its output, which lists various packages and their versions. The notebook title is 'DIP - HW1: 2D_signals_and_transforms' by 'Mohammad Amin Kiani 4043644008'.

Release notes:

- Google Colab's marketing site (<http://colab.google>) has a fresh new look!
- Added support for Nvidia RTX Pro 6000 Blackwell Server Edition GPUs on G4 machines. With a peak rating of 960 BF16 TFLOPs (~50% more than the A100-80G) and 96 GB of VRAM (20% more than the A100-80G), it's the most efficient high-performance GPU we've ever offered. Select "G4 GPU" in the runtime type menu.
- Colab's new Open Source MCP Server gives your local agents programmatic access to control the entire notebook development lifecycle. Use your local agent to build, run, and visualize in the browser. ([GitHub Repo](https://github.com/GoogleCloudPlatform/colab))
- Python package upgrades
 - accelerate 1.12.0 → 1.13.0
 - bigframes 2.35.0 → 2.38.0
 - blis 0.4.0 → 0.4.1
 - diffusers 0.36.0 → 0.37.0
 - google-adk 1.25.1 → 1.27.1
 - google-genai 1.64.0 → 1.67.0
 - h5py 3.15.1 → 3.16.0
 - huggingface_hub 1.4.1 → 1.7.1
 - kaggle 1.7.4.5 → 2.0.0
 - kagglehub 0.3.13 → 1.0.0
 - keras 3.10.0 → 3.13.2
 - keras-hub 0.21.1 → 0.26.0
 - keras-nlp 0.21.1 → 0.26.0
 - narwhals 2.17.0 → 2.18.0
 - openai 2.23.0 → 2.28.0





نتیجه	تابع
فرکانس بالا زیاد	Rectangle
فقط فرکانس پایین	Gaussian
بین دو مورد بالا	Pyramid
انرژی متمرکز در مرکز	Peak
smooth	Exponential

- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] Textbook - DIP Gonzalez 4th Edition